Window Functions, Explained: A Beginner's Series

Oct 7, 2026 · @SHERIF MOHAMED

About this series

Window functions let one query show a row's detail and its group's totals side by side. This four-part series builds that idea from zero, one small step per post.

It is written for people who already know SELECT, WHERE, GROUP BY, aggregates and joins, but have never used `OVER`. Every example runs on Pagila, the PostgreSQL version of the Sakila DVD-rental sample database. The main table is `payment`: one row per payment, with `payment_id`, `customer_id`, `amount` and `payment_date`.

Part	Title	You will learn
1	Detail vs context	Why GROUP BY can't answer some questions, and what a window is
2	Your first window	`OVER ()` and `PARTITION BY`, step by step
3	Trust, but verify	Proving a window result with a second query
4	Using window values	Calculations, order of execution, filtering with CTEs

Each part ends with short exercises. Try them before reading the next part.

Part 1 · Detail vs context: why GROUP BY isn't enough

Some business questions need a single row and its group's total on the same line. GROUP BY can't do that, and window functions exist to fill that gap.

A question GROUP BY can't answer

Take this request: "For each payment, what share of that customer's total spending is it?"

Every result row needs two things:

- **Detail:** this one payment's amount.
- **Context:** the customer's total across all their payments.

What GROUP BY does to your rows

GROUP BY makes one row per group. That is why every column in SELECT must either be in GROUP BY or sit inside an aggregate.

```
SELECT customer_id,
       COUNT(*)    AS payment_count,
       SUM(amount) AS total_paid
FROM payment
GROUP BY customer_id;
```

If a customer has 30 payments, they collapse into one row. There is no room for 30 different `payment_id` values, so the detail is gone. Try adding `payment_id` to that SELECT and PostgreSQL refuses to run it.

A plain SELECT has the opposite problem. It keeps every payment but knows nothing about the customer's total.

The word for this: grain

The **grain** of a result is what one row represents. `SELECT * FROM payment` has a grain of one row per payment. The GROUP BY query above has a grain of one row per customer.

GROUP BY changes the grain. That is exactly what you want for a summary report, and exactly what you don't want when you need detail and context together.

The idea of a window

A window function keeps the grain. For each row, it looks at a set of related rows, computes something over them, and writes the answer beside the row. No rows disappear.

Picture one customer's 30 payments in a spreadsheet. For each row, you highlight all 30 rows of that customer, add them up, and write the total in a new column. Every row gets the same total, and all 30 rows are still there. That highlighted block is the **window**.

payment_id	customer_id	amount	customer_total
101	1	2.99	10.97

payment_id	customer_id	amount	customer_total
102	1	0.99	10.97
103	1	6.99	10.97
104	2	4.99	9.98
105	2	4.99	9.98

The values are illustrative. The shape is the point: one row per payment, with the customer's total repeated on each.

Exercises

1. Write the GROUP BY query for total revenue and payment count per customer, top 5 by revenue.
2. In one sentence, describe the grain of `payment` and the grain of your query.
3. Add `payment_id` to your SELECT and read the error message. Explain why PostgreSQL refuses.

Part 2 · Your first window: OVER () and PARTITION BY

Any aggregate becomes a window function when you add `OVER (...)` after it. What goes inside the brackets decides which rows form the window.

The shape of a window function

```
SUM(amount)  OVER (  PARTITION BY customer_id  )
     |                        |
 what to compute      which rows belong together
```

The left side is an ordinary aggregate: `SUM`, `AVG`, `COUNT`, `MIN` or `MAX`. The `OVER` clause turns it into a column that sits beside each row instead of collapsing them.

Step 1: a window over everything

Empty brackets, `OVER ()`, mean the window is the whole result.

```
SELECT payment_id, customer_id, amount,
       SUM(amount) OVER () AS total_revenue
FROM payment
ORDER BY payment_id
LIMIT 10;
```

Every row shows the same grand total. It isn't useful yet, but it shows the mechanics: the aggregate became a column, and no rows collapsed.

Step 2: split the window with PARTITION BY

`PARTITION BY` splits the rows into groups, just like GROUP BY. The difference is that it doesn't collapse them. Each row's window is only the rows in its own group.

```
SELECT payment_id, customer_id, amount,
       SUM(amount) OVER (PARTITION BY customer_id) AS customer_total
FROM payment
ORDER BY customer_id, payment_id
LIMIT 40;
```

In the output, `customer_total` repeats on every row of a customer, then changes when `customer_id` changes.

Step 3: the other aggregates work the same way

```
SELECT payment_id, customer_id, amount,
       ROUND(AVG(amount) OVER (PARTITION BY customer_id), 2) AS customer_avg,
       COUNT(*)          OVER (PARTITION BY customer_id)     AS
customer_payments,
       MAX(amount)       OVER (PARTITION BY customer_id)     AS customer_max
FROM payment
ORDER BY customer_id, payment_id;
```

Two habits are worth building from the first query:

- **Round averages.** `AVG` on money returns long decimals like 4.2033333333. Wrap it in `ROUND(..., 2)`.
- **Always end with ORDER BY.** Without it, PostgreSQL returns rows in whatever order is cheapest, and that can change between runs. The window values stay correct, but the output is hard to read and hard to compare.

PARTITION BY vs GROUP BY in one sentence

GROUP BY changes the grain to one row per group. PARTITION BY defines the same groups for a calculation but keeps the original grain, so detail and group-level values sit on the same row.

Exercises

1. For each payment, show the amount, the customer's average payment and the difference between the two.
2. Show each payment with the total number of payments that customer made.
3. Show each payment with the customer's largest payment beside it.

Part 3 · Trust, but verify: proving a window result

A query that runs is not the same as a query that's right. Professionals check important numbers with a second, different query, and window functions are a good place to start that habit.

The "old way": join to a grouped subquery

Before window functions, the standard way to put a group total beside each row was two steps. First summarise in a subquery, then join the summary back to the detail rows.

```
SELECT p.payment_id, p.customer_id, p.amount, t.customer_total
FROM payment p
JOIN (SELECT customer_id, SUM(amount) AS customer_total
      FROM payment
      GROUP BY customer_id) t
  ON t.customer_id = p.customer_id
ORDER BY p.customer_id, p.payment_id;
```

This returns the same numbers as the `PARTITION BY` version from Part 2. The window version is shorter and reads the table once, but the join version is built completely differently. That makes it a good independent check.

Checking one customer by eye

The simplest check is to filter both queries to one customer and compare the outputs side by side. Add `WHERE customer_id = 1` to the window query and `WHERE p.customer_id = 1` to the join query, then compare the rows.

This works, but it only proves one customer. The numbers for the other 598 could still be wrong.

Checking every row with EXCEPT

`EXCEPT` returns the rows of the first query that don't appear in the second. If two queries agree completely, `EXCEPT` returns **zero rows**.

```
(SELECT payment_id,
        ROUND(AVG(amount) OVER (PARTITION BY customer_id), 2) AS avg_amt
 FROM payment)
EXCEPT
(SELECT p.payment_id, ROUND(av.avg_amt, 2)
 FROM payment p
 JOIN (SELECT customer_id, AVG(amount) AS avg_amt
       FROM payment
       GROUP BY customer_id) av
   ON av.customer_id = p.customer_id);
```

Three details make this comparison fair:

- Both sides return the **same columns in the same order**.
- Both sides **round the same way**, so tiny decimal differences don't show up as false mismatches.
- For a full check, run it **in both directions** (A EXCEPT B, then B EXCEPT A). Zero rows each way means the results are identical.

Why this matters beyond practice

In analytics work, a wrong number in a dashboard costs trust that takes months to rebuild. Proving a result with a second method is the same idea that automated data tests are built on later.

Exercises

1. Verify Part 2's exercise 1 (the customer average) with the join method, for customer 1 only.
2. Write the `EXCEPT` check in both directions for the same query. Confirm both return zero rows.
3. Break it on purpose: round one side to 1 decimal place and the other to 2. Run the check again and explain what you see.

Part 4 · Using window values: calculations, order of execution and CTEs

A window value behaves like any other column in SELECT, so you can do arithmetic with it. Filtering on it is different, and understanding why teaches you how PostgreSQL runs a query.

Answering the original question

Part 1 opened with "what share of the customer's total is each payment?" Divide the payment by the window total:

```
SELECT payment_id, customer_id, amount,
       SUM(amount) OVER (PARTITION BY customer_id) AS customer_total,
       ROUND(100.0 * amount
             / SUM(amount) OVER (PARTITION BY customer_id), 2) AS
pct_of_customer
FROM payment
ORDER BY customer_id, payment_id;
```

The `100.0` makes the result a percentage, and the decimal point keeps the maths in decimals.

Why WHERE can't see a window

Now try to keep only payments worth more than 10% of the customer's total, with `WHERE pct_of_customer > 10`. PostgreSQL refuses.

The reason is the **logical order** in which a query is processed. It isn't the order you write the clauses in:

1. `FROM`: pick the tables and join them.

2. `WHERE`: filter individual rows.
3. `GROUP BY`: form groups.
4. `HAVING`: filter groups.
5. **Window functions**: calculated here.
6. `SELECT`: choose and name the output columns.
7. `ORDER BY`: sort the result.

`WHERE` runs at step 2, and windows don't exist until step 5. When WHERE runs, there is no `pct_of_customer` to test.

The fix: a CTE

A **CTE** (Common Table Expression) is a named step written with `WITH`. Read it as "first build this table, then query it." Compute the window inside the CTE, and filter in the query outside it:

```
WITH payment_share AS (
    SELECT payment_id, customer_id, amount,
           ROUND(100.0 * amount
                 / SUM(amount) OVER (PARTITION BY customer_id), 2) AS
pct_of_customer
    FROM payment
)
SELECT payment_id, customer_id, amount, pct_of_customer
FROM payment_share
WHERE pct_of_customer > 10
ORDER BY pct_of_customer DESC;
```

By the time the outer WHERE runs, the CTE has already finished, so `pct_of_customer` is just an ordinary column.

Naming habits for CTEs

- **Name the CTE after what the table is**, such as `payment_share` or `payments_with_max`.
- **Name columns after what the value is**, such as `pct_of_customer` or `max_amount`.
- **Never give a CTE and a column the same name.** It makes the final query confusing to read.

- **List columns explicitly** in the final SELECT instead of `SELECT *`. Readers see exactly what the query returns.

A trap to notice: ties

To find each customer's biggest payment, keep rows where `amount` equals the window `MAX`:

```
WITH payments_with_max AS (
    SELECT payment_id, customer_id, amount,
           MAX(amount) OVER (PARTITION BY customer_id) AS max_amount
    FROM payment
)
SELECT payment_id, customer_id, amount
FROM payments_with_max
WHERE amount = max_amount
ORDER BY customer_id, payment_id;
```

Some customers appear more than once. If a customer made two payments of 11.99 and that's their maximum, both rows match. "Equals the max" keeps **ties**, rows with the same value.

Sometimes that is what you want. When you need exactly one row per customer, you need a different tool: ranking functions.

What comes next

The next series covers `ROW_NUMBER`, `RANK` and `DENSE_RANK`. They number rows inside each group in a chosen order, and they give you precise control over ties.

Exercises

1. List payments worth more than 20% of their customer's total, largest share first.
2. Find each customer's smallest payment with `MIN ... OVER` and a CTE. Do any customers appear more than once? Why?
3. Write the logical order of execution from memory. Then explain in one sentence why a window value can't be used in WHERE.